

一、项目翻译

项目 1: 设计并验证信源编码

- **任务:** 为文本 (Text) 设计一种无损信源编码 (Lossy compression 改为 Lossless compression), 编写程序实现它, 并验证其编码效率 (即无损压缩)。
- **流程图含义:**
$$D \xrightarrow{\text{压缩 (Compress)}} C \xrightarrow{\text{解压 (Decompress)}} D$$

注: 因为是无损压缩, 最终还原的文档 D 必须与初始文档 D 完全一模一样, 不能有任何丢字或错字。

二、4人小组分工建议

由于这个项目不需要模拟复杂的信道噪声 (BSC/BEC), 也不需要处理图像矩阵, 它的核心难点在于对文本字符的统计、概率树的构建、以及高效的二进制位流 (Bitstream) 读写。

针对你们的 4 人小组, 建议采用以下分工, 让每个人都有扎实的代码或理论产出:

1. 同学 A: 频率统计与数据结构构建 (Data & Tree Builder)

- **核心任务:** 负责读取文本并构建核心数据结构。
- **具体工作:**
 - 编写代码读取输入的 .txt 文本文件, 统计每个字符 (字母、标点、空格、换行符) 出现的频率/概率。
 - 如果选择霍夫曼编码 (Huffman Coding), 负责实现最小堆/优先队列, 并根据字符频率逆向组合, 构建出霍夫曼树 (Huffman Tree)。
 - 将树形结构转化为“字符 $\rightarrow$ 二进制字符串 (如 'a' $\rightarrow$ '010')”的密码本 (Mapping Table), 提供给同学 B。

2. 同学 B: 二进制流压缩编码器 (Bitstream Encoder)

- **核心任务:** 负责将文本转换成真正的压缩文件。
- **具体工作:**
 - 接收同学 A 的密码本, 将原始文本中的每个字符替换为对应的二进制字符串。
 - **难点与核心:** 不能直接存成普通的字符串, 必须实现位操作 (Bit manipulation)。每凑齐 8 个比特 (bit) 就打包成一个字节 (byte) 写入二进制文件 (即图中的 Compressed Document C)。
 - 负责在压缩文件头部写入“解压所需的字典/树结构信息”, 以便同学 C 能够跨文件解压。

3. 同学 C: 二进制流解压译码器 (Bitstream Decoder)

- **核心任务:** 负责从压缩文件中完美还原文本。
- **具体工作:**
 - 读取同学 B 生成的二进制压缩文件, 首先解析出文件头的字典或树结构。
 - 依次读取紧随其后的二进制位 (0 和 1), 在霍夫曼树中从根节点开始向下查找, 直到到达叶子节点, 解出对应的原文字符。
 - 将还原的字符输出为新文本文件 (Reconstituted Document D), 供同学 D 进行无损验证。

4. 同学 D：自动化测试、效率验证与英文报告（QA & Lead Writer）

- **核心任务：** 验证无损一致性、计算指标、统筹报告。
- **具体工作：**
 - **验证无损（Verification）：** 编写脚本对比初始文档 D 和重构文档 D' 的每一个字符（或者对比两者的 MD5 校验码），确保 $D == D'$ （误码率必须绝对为 0）。
 - **验证编码效率（Coding Efficiency）：** * 计算**压缩比（Compression Ratio）** = 压缩后文件大小 / 原始文件大小。
 - 计算文本的**信息熵（Entropy）**： $H(X) = -\sum p(x) \log_2 p(x)$ 。
 - 计算**平均码长（Average Code Length）**，并与信息熵进行对比，计算出**编码效率**（越接近 100% 说明设计得越好）。
 - 主笔撰写英文实验报告。

三、核心思路与算法推荐

对于文本的无损压缩，有以下两种主流的经典算法思路供你们选择：

方案一：霍夫曼编码（Huffman Coding）—— 强烈推荐

- **为什么选它：** 概念极其经典，非常适合用来完成信息论的作业。
- **核心逻辑：** 出现频率高的字符用短编码（比如 e 用 01），出现频率低的字符用长编码（比如 z 用 11010）。因为任何一个编码都不是另一个编码的前缀（前缀码），所以解压时不会产生歧义。

方案二：LZ77 / LZW 字典编码 —— 进阶高分选项

- **为什么选它：** 这是类似 gzip 或 zip 软件的核心原理。
- **核心逻辑：** 它不依赖字符的单字概率，而是找“重复的词组”。比如文本里反复出现 information，它就会在第一次出现后记录一个指针（向前数第 X 个位置，长度为 Y），通过动态构建字典来压缩空间。

四、快速起步硬性工具

- **编程语言：** 依然强烈推荐 **Python**。Python 处理文件读写非常方便。
- **硬性库/函数：**
 - Python 内置的 collections.Counter：可以借给同学 A 一键统计字符频率。
 - Python 内置的 heapq：可以用来轻松构建霍夫曼树的优先队列。
 - 读写二进制文件时，使用 open('filename', 'wb')（写二进制）和 open('filename', 'rb')（读二进制）。

你们可以像 Project 2 一样，在 GitHub 上由 main 分支拉出 A、B、C 三个开发分支，各自写完后由 D 同学合并，在本地准备几篇长篇的英文文章（如英文小说、新闻）作为输入文档 D 来测试你们的系统。